TOPIC 3 : DIVIDE AND CONQUER

31. Write a Program to find both maximum and minimum values in array using Divide and Conquer. Input: N = 8, a[] = {5,7,3,4,9,12,6,2} -> Output: Min = 2, Max = 12 Input: N = 9, a[] = {1,3,5,7,9,11,13,15,17} -> Output: Min = 1, Max = 17 Input: N = 10, a[] = {22,34,35,36,43,67,12,13,15,17} -> Output: Min = 12, Max = 67

Aim

To write a Python program using Divide and Conquer recursive splitting to find minimum and maximum elements in an array.

Algorithm

1. Start.
2. Base cases: if len == 1, return (a[0], a[0]); if len == 2, return (min(a[0],a[1]), max(a[0],a[1])).
3. Divide array into left and right halves at mid = (low + high) // 2.
4. Recursively call find_min_max on left and right sub-arrays.
5. Combine results: return (min(left_min, right_min), max(left_max, right_max)).
6. Stop.

main.py Run

```python
def find_min_max(arr, low, high):
    if low == high:
        return arr[low], arr[low]
    if high == low + 1:
        if arr[low] < arr[high]:
            return arr[low], arr[high]
        else:
            return arr[high], arr[low]

    mid = (low + high) // 2
    min1, max1 = find_min_max(arr, low, mid)
    min2, max2 = find_min_max(arr, mid + 1, high)

    return min(min1, min2), max(max1, max2)

arr1 = [5, 7, 3, 4, 9, 12, 6, 2]
mn1, mx1 = find_min_max(arr1, 0, len(arr1) - 1)
print(f"Min = {mn1}, Max = {mx1}")

arr2 = [1, 3, 5, 7, 9, 11, 13, 15, 17]
mn2, mx2 = find_min_max(arr2, 0, len(arr2) - 1)
print(f"Min = {mn2}, Max = {mx2}")
```

Output

```
Min = 2, Max = 12
Min = 1, Max = 17
```

Time Complexity

O(n) time complexity using 3n/2 - 2 comparisons.

Space Complexity

O(log n) call stack depth.

32. Consider an array of integers sorted in ascending order: 2,4,6,8,10,12,14,18. Write a program to find minimum and maximum values. Input: N = 8, a[] = {2,4,6,8,10,12,14,18} -> Output: Min = 2, Max = 18 Input: N = 9, a[] = {11,13,15,17,19,21,23,35,37} -> Output: Min = 11, Max = 37

Aim

To write a Python program to extract boundary minimum and maximum values in a sorted array.

Algorithm

1. Start.
2. Read sorted array.
3. Minimum element is arr[0].
4. Maximum element is arr[-1].
5. Return arr[0] and arr[-1].
6. Stop.

main.py Run

```
def get_min_max_sorted(arr):
    if not arr: return None, None
    return arr[0], arr[-1]

arr1 = [2, 4, 6, 8, 10, 12, 14, 18]
mn1, mx1 = get_min_max_sorted(arr1)
print(f"Min = {mn1}, Max = {mx1}")

arr2 = [11, 13, 15, 17, 19, 21, 23, 35, 37]
mn2, mx2 = get_min_max_sorted(arr2)
print(f"Min = {mn2}, Max = {mx2}")
```

Output

```
Min = 2, Max = 18
Min = 11, Max = 37
```

Time Complexity

O(1) constant time bound lookup.

Space Complexity

O(1) auxiliary space.

33. You are given an unsorted array 31,23,35,27,11,21,15,28. Write a program for Merge Sort. Input: N = 8, a[] = {31,23,35,27,11,21,15,28} -> Output: 11,15,21,23,27,28,31,35 Input: N = 10, a[] = {22,34,25,36,43,67,52,13,65,17} -> Output: 13,17,22,25,34,36,43,52,65,67

Aim

To write a Python program implementing Merge Sort using divide-and-conquer principles.

Algorithm

1. Start.
2. Divide array into two equal halves.
3. Recursively sort left and right halves.
4. Merge the two sorted halves.
5. Return fully sorted array.
6. Stop.

main.py Run

```python
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

a1 = [31, 23, 35, 27, 11, 21, 15, 28]
print(",".join(map(str, merge_sort(a1))))
```

Output

```
11,15,21,23,27,28,31,35
```

Time Complexity

O(n log n) across all input distributions.

Space Complexity

O(n) space for merging sub-arrays.

34. Implement Merge Sort algorithm and modify implementation to count the number of comparisons made during sorting. Input: N = 8, a[] = {12,4,78,23,45,67,89,1} -> Output: 1,4,12,23,45,67,78,89 Input: N = 7, a[] = {38,27,43,3,9,82,10} -> Output: 3,9,10,27,38,43,82

Aim

To write a Python program to count exact total comparisons executed by Merge Sort.

Algorithm

1. Start.
2. Global or pass-by-reference comparison counter.
3. Increment comparison count on every key comparison during merge step.
4. Return sorted list and total comparison count.
5. Stop.

main.py

Run

```python
def merge_sort_count(arr):
    comparisons = 0

    def sort(sub_arr):
        nonlocal comparisons
        if len(sub_arr) <= 1:
            return sub_arr
        mid = len(sub_arr) // 2
        left = sort(sub_arr[:mid])
        right = sort(sub_arr[mid:])

        merged = []
        i = j = 0
        while i < len(left) and j < len(right):
            comparisons += 1
            if left[i] < right[j]:
                merged.append(left[i])
                i += 1
            else:
                merged.append(right[j])
                j += 1
        merged.extend(left[i:])
        merged.extend(right[j:])
        return merged

    sorted_arr = sort(arr)
    return sorted_arr, comparisons

res, cnt = merge_sort_count([12, 4, 78, 23, 45, 67, 89, 1])
print("Sorted:", res, "Comparisons:", cnt)
```

Output

```
Sorted: [1, 4, 12, 23, 45, 67, 78, 89] Comparisons: 16
```

Time Complexity

O(n log n) time complexity.

Space Complexity

O(n) auxiliary array space.

35. Given unsorted array 10,16,8,12,15,6,3,9,5. Perform Quick Sort choosing first element as pivot. Input: N = 9, a[] = {10,16,8,12,15,6,3,9,5} -> Output: 3,5,6,8,9,10,12,15,16

Aim

To write a Python program to perform Quick Sort choosing the first element as pivot.

Algorithm

1. Start.
2. Pick pivot = arr[0].
3. Partition remaining array into elements < pivot and elements >= pivot.
4. Recursively apply Quick Sort on left and right partitions.
5. Combine: left_sorted + [pivot] + right_sorted.
6. Stop.

main.py Run

```python
def quick_sort_first(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[0]
    left = [x for x in arr[1:] if x < pivot]
    right = [x for x in arr[1:] if x >= pivot]
    return quick_sort_first(left) + [pivot] + quick_sort_first(right)

arr = [10, 16, 8, 12, 15, 6, 3, 9, 5]
print(",".join(map(str, quick_sort_first(arr))))
```

Output

```
3,5,6,8,9,10,12,15,16
```

Time Complexity

Average: O(n log n), Worst Case: $O(n^2)$ when array already sorted.

Space Complexity

O(n) space for partition sub-lists.

36. Implement Quick Sort choosing middle element as pivot. Input: N = 8, a[] = {19,72,35,46,58,91,22,31} -> Output: 19,22,31,35,46,58,72,91

Aim

To write a Python program to execute Quick Sort with middle element as partition pivot.

Algorithm

1. Start.
2. Pick pivot = arr[len(arr) // 2].
3. Partition array into smaller, equal, and larger sub-arrays.
4. Recursively sort smaller and larger parts.
5. Concatenate results.
6. Stop.

main.py Run

```
def quick_sort_middle(arr):
    if len(arr) <= 1:
        return arr
    mid_idx = len(arr) // 2
    pivot = arr[mid_idx]
    left = [x for i, x in enumerate(arr) if x < pivot or (x == pivot and i != mid_idx and i < mid
    right = [x for i, x in enumerate(arr) if x > pivot or (x == pivot and i != mid_idx and i > mi
    return quick_sort_middle(left) + [pivot] + quick_sort_middle(right)

arr = [19, 72, 35, 46, 58, 91, 22, 31]
print(",".join(map(str, quick_sort_middle(arr))))
```

Output

```
19,22,31,35,46,58,72,91
```

Time Complexity

Average: O(n log n), Worst Case: O(n log n) with balanced middle pivot.

Space Complexity

O(n) partition space.

37. Implement Binary Search to find search key 20 in array 5,10,15,20,25,30,35,40,45. Count the number of comparisons made. Input: N = 9, a[] = {5,10,15,20,25,30,35,40,45}, search key = 20 -> Output position: 4

Aim

To write a Python program to search for a key in a sorted array using Binary Search while tracking comparison count.

Algorithm

1. Start.
2. Initialize low = 0, high = n - 1, count = 0.
3. Loop while low <= high:
4. mid = (low + high) // 2
5. count += 1
6. If arr[mid] == key, return mid, count.
7. If key < arr[mid]: high = mid - 1
8. Else: low = mid + 1.
9. Stop.

main.py Run

```python
def binary_search_count(arr, key):
    low, high = 0, len(arr) - 1
    comps = 0
    while low <= high:
        comps += 1
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid + 1, comps
        elif key < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1
    return -1, comps

pos, c = binary_search_count([5,10,15,20,25,30,35,40,45], 20)
print(f"Output position: {pos} (Comparisons: {c})")
```

Output

```
Output position: 4 (Comparisons: 4)
```

Time Complexity

O(log n) logarithmic time search.

Space Complexity

O(1) space complexity.

38. Given sorted array 3,9,14,19,25,31,42,47,53, find position of 31 showing mid-point calculations. Input: N = 9, a[] = {3,9,14,19,25,31,42,47,53}, search key = 31 -> Output: 6

Aim

To write a Python program to trace step-by-step mid-point calculations during Binary Search.

Algorithm

1. Start.
2. Set low = 0, high = n - 1.
3. Calculate mid = (low + high) // 2 at each step.
4. Print step state (low, high, mid, arr[mid]).
5. Adjust search boundaries until key found.
6. Stop.

main.py Run

```python
def trace_binary_search(arr, key):
    low, high = 0, len(arr) - 1
    step = 1
    while low <= high:
        mid = (low + high) // 2
        print(f"Step {step}: low={low}, high={high}, mid={mid}, arr[mid]={arr[mid]}")
        if arr[mid] == key:
            return mid + 1
        elif key < arr[mid]:
            high = mid - 1
        else:
            low = mid + 1
        step += 1
    return -1

pos = trace_binary_search([3,9,14,19,25,31,42,47,53], 31)
print("Position:", pos)
```

Output

```
Step 1: low=0, high=8, mid=4, arr[mid]=25
Step 2: low=5, high=8, mid=6, arr[mid]=42
Step 3: low=5, high=5, mid=5, arr[mid]=31
Position: 6
```

Time Complexity

O(log n) binary search bound traversal.

Space Complexity

O(1) space complexity.

39. Given an array of points where points[i] = [xi, yi] and integer k, return the k closest points to origin (0,0). Input: points = [[1,3],[-2,2],[5,8],[0,1]], k = 2 -> Output: [[-2,2],[0,1]] Input: points = [[1,3],[-2,2]], k = 1 -> Output: [[-2,2]]

Aim

To write a Python program using Divide and Conquer / QuickSelect to find K closest points to origin.

Algorithm

1. Start.
2. Compute squared Euclidean distance $x^2 + y^2$ for each point.
3. Sort points based on distance (or use QuickSelect divide and conquer).
4. Return first k points from sorted list.
5. Stop.

main.py Run

```
1  def kClosest(points, k):
2      points.sort(key=lambda p: p[0]**2 + p[1]**2)
3      return points[:k]
4
5  print(kClosest([[1,3],[-2,2],[5,8],[0,1]], 2))
6  print(kClosest([[1,3],[-2,2]], 1))
```

Output

```
[[0, 1], [-2, 2]]
[[-2, 2]]
```

Time Complexity

O(n log n) with sorting, or O(n) average with QuickSelect divide-and-conquer.

Space Complexity

O(k) output list space.

40. Given four lists A, B, C, D of integer values, compute how many tuples (i, j, k, l) exist such that A[i] + B[j] + C[k] + D[l] = 0. Input: A = [1,2], B = [-2,-1], C = [-1,2], D = [0,2] -> Output: 2 Input: A = [0], B = [0], C = [0], D = [0] -> Output: 1

Aim

To write a Python program using Divide and Conquer / Hash Map partitioning to solve 4Sum II in $O(n^2)$ time.

Algorithm

1. Start.
2. Divide problem into two sub-problems of size 2 (A+B and C+D).
3. Compute frequencies of all pair sums A[i] + B[j] and store in hash map count_ab.
4. Iterate all pair sums C[k] + D[l] and look up -(C[k] + D[l]) in count_ab.
5. Add matching frequencies to total tuple count.
6. Return total count.
7. Stop.

main.py Run

```python
from collections import Counter

def fourSumCount(A, B, C, D):
    ab_sum = Counter(a + b for a in A for b in B)
    return sum(ab_sum[-(c + d)] for c in C for d in D)

print(fourSumCount([1, 2], [-2, -1], [-1, 2], [0, 2]))
print(fourSumCount([0], [0], [0], [0]))
```

Output

```
2
1
```

Time Complexity

$O(n^2)$ time complexity.

Space Complexity

$O(n^2)$ hash map space for storing pair sums of A and B.